

C++ High-Performance Task & Event Processing Engine

A complete architectural blueprint for building a networked, persistent, multithreaded systems engine — inspired by Redis, Celery, and Apache Kafka — built from scratch in C++17 with full crash recovery and TCP networking.

Language	C++17
Build System	CMake 3.16+
Timeline	11–13 Weeks
Components	5 Subsystems
Testing	ThreadSanitizer + 60s Stress

Inspired by [Redis](#) · [Celery](#) · [Apache Kafka](#)

Table of Contents

1	Project Overview
2	Architecture Philosophy
3	Complete File & Folder Structure
4	Component 1 — KV Store (Redis-Inspired)
5	Component 2 — Task Queue (Celery-Inspired)
6	Component 3 — Pub-Sub System (Kafka-Inspired)
7	Component 4 — TCP Server (Networking Layer)
8	Component 5 — WAL & Crash Recovery
9	Component 6 — Engine Core
10	Build System — CMake
11	Testing Strategy
12	Benchmarking Strategy
13	Phase-by-Phase Build Order
14	Key Design Decisions & Why
15	What Every File Does
16	Interview Preparation

1 — Project Overview

What You Are Building

A single-process, multithreaded server in C++ combining five distinct subsystems into one unified engine. This is not a tutorial clone — it is a genuine systems project that mirrors the internal architecture of production infrastructure tools used at scale by companies like Netflix, Uber, and LinkedIn.

Subsystem	Real-World Inspiration	Core Concept
Key-Value Store + TTL	Redis	Fast in-memory storage with automatic expiry
Task Queue + Retry	Celery	Background job execution with fault tolerance
Pub-Sub Messaging	Apache Kafka	Async event broadcasting to subscribers
TCP Server	Redis protocol	Networked access from any language or client
Write-Ahead Log	PostgreSQL / Redis AOF	Data persistence and crash recovery

Why This Project Matters

Most backend systems you will ever work on professionally use these exact primitives — caching, background jobs, and event messaging. Understanding how they work internally, not just how to use them, puts you in a completely different category as an engineer.

- Concurrency and thread safety at the hardware level
- Memory ownership and lifetime management — zero garbage collector
- Network protocol design over raw POSIX TCP sockets
- Data persistence and crash recovery via write-ahead logging
- Performance measurement with nanosecond-precision benchmarks

What This Is NOT

This is not meant to replace Redis or Kafka. It is an educational systems project designed to teach you how these systems think and why they make the design choices they do. The goal is deep understanding, not production deployment.

2 — Architecture Philosophy

The Three Laws of This Project

Law 1 — One component at a time.

Never build Component 2 until Component 1 is fully working, tested, and benchmarked. Concurrency bugs compound across components and become impossible to debug when multiple unverified systems are in play.

Law 2 — ThreadSanitizer from day one.

Every component is compiled and tested with -fsanitize=thread from the first line of code. Race conditions caught early cost minutes. Race conditions caught late cost days of blind debugging.

Law 3 — Stress test before moving on.

Every component has a stress test hammering it with multiple threads for at least 30 seconds before you write the next component. If it survives that, it is ready.

How the Five Components Interact

Each component is independently usable. The Engine Core owns and wires them. The TCP Server exposes everything over the network. Data flows top-down: client commands enter through the network layer, are routed by the Engine, and land in the appropriate subsystem.

TCP SERVER	Accepts client connections, reads commands, sends responses
ENGINE CORE	Routes commands to the correct subsystem, manages startup order
KV STORE	In-memory hash map with TTL — reads and writes key-value pairs
TASK QUEUE	Thread pool + retry logic — executes background jobs
PUB-SUB BUS	Topic routing — delivers events to all subscribers asynchronously
WAL / DISK	Appends every write to disk before applying to memory

3 — Complete File & Folder Structure

Every file has a single, clear responsibility. Components are isolated in their own directories. This structure mirrors how real production codebases are organized and immediately signals architectural clarity to any engineer reading the repository.

```

cpp-engine/
  ■■■ CMakeLists.txt          <- Root build file
  ■■■ README.md              <- Overview + benchmark results
  ■■■ main.cpp                <- Entry point: starts TCP server
  ■
  ■■■ src/
    ■ ■■■ kvstore/
      ■ ■■■ KVStore.h / .cpp  <- Core get/set/del logic
      ■ ■■■ TTLManager.h / .cpp <- Background janitor thread
      ■ ■■■ KVEntry.h         <- {value, expiry_time} struct
    ■ ■■■ taskqueue/
      ■ ■■■ ThreadPool.h / .cpp <- Worker thread pool
      ■ ■■■ Task.h            <- Task struct + state enum
      ■ ■■■ TaskQueue.h / .cpp <- Queue with retry + backoff
      ■ ■■■ RetryPolicy.h     <- Backoff configuration
    ■ ■■■ pubsub/
      ■ ■■■ EventBus.h / .cpp  <- Topic routing + async dispatch
      ■ ■■■ Message.h         <- Message struct
    ■ ■■■ network/
      ■ ■■■ TCPServer.h / .cpp <- Socket accept loop
      ■ ■■■ ClientHandler.h / .cpp <- Per-client thread handler
      ■ ■■■ CommandParser.h    <- Parse raw strings to commands
    ■ ■■■ persistence/
      ■ ■■■ WAL.h / .cpp       <- Append + fsync logic
      ■ ■■■ Recovery.cpp       <- Replay WAL on startup
    ■ ■■■ engine/
      ■ ■■■ Engine.h / .cpp    <- Wires all components together
    ■
  ■■■ tests/
    ■ ■■■ test_kvstore.cpp     <- Unit: get/set/del/TTL
    ■ ■■■ test_threadpool.cpp  <- Unit: task dispatch
    ■ ■■■ test_pubsub.cpp      <- Unit: subscribe/publish
    ■ ■■■ test_wal.cpp         <- Unit: append/recover
    ■ ■■■ stress_test.cpp      <- 20 threads x 60 seconds
    ■
  ■■■ benchmarks/
    ■ ■■■ bench_kvstore.cpp    <- Ops/sec, latency
    ■ ■■■ bench_taskqueue.cpp  <- Throughput
    ■ ■■■ bench_pubsub.cpp     <- Delivery lag
    ■
  ■■■ demos/
    ■ ■■■ demo_kvstore.cpp     <- Interactive TTL demo
    ■ ■■■ demo_taskqueue.cpp   <- Retry + backoff demo
    ■ ■■■ demo_full.cpp        <- All 3 systems live
    ■
  ■■■ docs/
    ■■■ ARCHITECTURE.md        <- System diagram + roles
    ■■■ DESIGN_DECISIONS.md    <- Why you chose X over Y
    ■■■ COMMANDS.md           <- TCP command reference
    ■■■ BENCHMARKS.md         <- Measured results

```

4 — Component 1 — KV Store (Redis-Inspired)

What It Is

An in-memory key-value store where any string key maps to any string value, with optional automatic expiry (TTL). Multiple threads can read and write simultaneously without corrupting data.

Core Concepts Required

<code>std::unordered_map</code>	The underlying data structure. O(1) average-case lookup, insert, delete.
<code>std::shared_mutex</code>	Allows multiple concurrent readers but only one writer at a time.
<code>std::shared_lock</code>	Used for GET operations. Multiple threads hold this simultaneously.
<code>std::unique_lock</code>	Used for SET/DEL. Exclusive — blocks until all readers release.
<code>std::chrono::steady_clock</code>	Monotonic clock for TTL. Never goes backwards even if system time changes.
<code>std::atomic<bool></code>	Thread-safe flag for the janitor thread run/stop state.

Two-Phase TTL Expiry — The Key Design

The TTL system uses a dual strategy deliberately. Lazy expiry checks on every GET — fast but leaks memory for keys never read again. Active expiry via a background janitor thread scans all keys every 100ms and deletes expired ones. Together they cover both cases. This is the exact same dual-strategy Redis uses internally.

Critical Insight — Two-Phase Deletion in `purgeExpired()`

The janitor must NOT hold a write lock during the full scan. Doing so blocks all reads for the entire scan duration — potentially 200ms+ on a large store. The correct approach: scan under a read lock first, collect expired keys, then switch to a write lock only for the brief deletion.

```
void KVStore::purgeExpired() {
    // Phase 1: collect expired keys under READ lock
    //          (normal reads continue unblocked during this)
    vector<string> toDelete;
    {
        shared_lock lock(mutex_);
        for (auto& [key, entry] : store_)
            if (entry.isExpired()) toDelete.push_back(key);
    }
    // Phase 2: delete under WRITE lock (brief exclusive window)
    if (!toDelete.empty()) {
        unique_lock lock(mutex_);
        for (auto& key : toDelete) store_.erase(key);
    }
}
```

Why this matters: This optimization cuts read latency by up to 40% under TTL-heavy workloads. It is the kind of design decision that makes interviewers lean forward — because it shows you thought about the system under real load, not just correctness.

5 — Component 2 — Task Queue (Celery-Inspired)

What It Is

A system that accepts work items (tasks) and executes them asynchronously using a pool of worker threads. Tasks can fail and be retried automatically with exponentially increasing delays. Every task has a trackable lifecycle state.

Task State Machine

PENDING	Task submitted to the queue, not yet picked up by a worker
RUNNING	Worker thread has picked up the task and is executing it
DONE	Task completed successfully — future resolved with result
FAILED	Task exhausted all retries — future carries the exception

The Worker Loop — Heart of the Thread Pool

```
void ThreadPool::workerLoop() {
    while (true) {
        function<void()> task;
        {
            unique_lock lock(queueMutex_);
            // Predicate prevents spurious wakeups
            cv_.wait(lock, [this] {
                return !taskQueue_.empty() || stop_.load();
            });
            if (stop_.load() && taskQueue_.empty()) return;
            task = move(taskQueue_.front());
            taskQueue_.pop();
        }
        task(); // Execute OUTSIDE the lock -- critical
    }
}
```

Critical detail: The task executes after the lock is released. Executing while holding the lock would prevent other workers from picking up tasks simultaneously, defeating the entire purpose of the thread pool. The predicate in `cv_.wait` prevents spurious wakeups — condition variables can wake randomly even without `notify()` being called.

Exponential Backoff with Jitter

When a task fails, it is not retried immediately. The delay grows exponentially and random jitter is added to prevent the thundering herd problem — a well-known distributed systems failure mode.

Without jitter:

```
Retry 1: sleep 200ms (base delay)
Retry 2: sleep 400ms (2x)
Retry 3: sleep 800ms (4x)
```

With jitter (+/- 20% random spread):

```
Retry 1: sleep 160ms - 240ms
Retry 2: sleep 320ms - 480ms
Retry 3: sleep 640ms - 960ms
```

Why jitter matters: Without it, 1000 tasks that all fail simultaneously will all retry at exactly T+200ms, creating a thundering herd that overwhelms the downstream service and prevents recovery. Jitter spreads retries across a time window, smoothing load. AWS, Google, and Stripe all document this pattern.

6 — Component 3 — Pub-Sub System (Kafka-Inspired)

What It Is

A message routing system where publishers send messages to named topics and any number of subscribers receive them asynchronously. Publishers never wait for subscribers to finish processing — they always return immediately.

Why Publishing Must Be Async

If subscriber callbacks run in the publisher's thread: a slow subscriber blocks the publisher and all other subscribers; a throwing callback crashes the publisher. The correct design dispatches each callback to the thread pool, isolating every subscriber from each other.

```
void EventBus::publish(const Message& msg) {
    vector<Callback> callbacks;

    // Phase 1: collect callbacks under READ lock
    {
        shared_lock lock(mutex_);
        auto it = topics_.find(msg.topic);
        if (it == topics_.end()) return;
        for (auto& sub : it->second)
            callbacks.push_back(sub.callback);
    }

    // Phase 2: dispatch each to thread pool
    //          publisher returns immediately after this loop
    for (auto& cb : callbacks)
        pool_.submit([cb, msg]() { cb(msg); });
}
```

Cross-Component Integration — KV Store + Pub-Sub

When the TTL janitor deletes an expired key, it publishes a **key.expired** event to the EventBus. Any subscriber can react — invalidate a secondary cache, log the event, trigger a data refresh. This is exactly how Redis keyspace notifications work, and it is one of the most visually impressive things to demo because it shows two components working together seamlessly.

7 — Component 4 — TCP Server (Networking Layer)

Why This Transforms the Project

Without networking, your engine is a C++ library — useful only if someone links against it. With networking, it becomes a service — accessible from Python, Go, JavaScript, a shell script, or any tool. A recruiter watching a demo sees a running server, not a library. That is a fundamentally different thing.

Command Protocol

A simple line-based text protocol deliberately similar to the Redis Serialization Protocol (RESP). Design inspiration you can explain clearly.

```
# KV Store commands
SET key value [EX milliseconds]
GET key
DEL key

# Task Queue commands
SUBMIT task_name [payload]
STATUS task_id

# Pub-Sub commands
SUBSCRIBE topic
PUBLISH topic message

# Responses
+OK                <- success
+VALUE Alice       <- value returned
-ERR Not found     <- error
```

Demo from Terminal (What Recruiters See)

```
# Terminal 1 — engine running
./engine --port=6380

# Terminal 2 — any client connecting
telnet localhost 6380
> SET user:1 Alice EX 5000
+OK
> GET user:1
+VALUE Alice
> SUBSCRIBE orders
+OK subscribed
```

8 — Component 5 — WAL & Crash Recovery

What It Is

Before any write operation modifies in-memory data, the operation is first written to a file on disk. On restart, the engine reads that file and replays every operation in order, reconstructing the exact in-memory state. This is the D in ACID — durability — and it is the fundamental mechanism behind every real database's persistence guarantee.

WAL File Format

Each line is one operation — human-readable, trivially parseable, inspectable with `cat wal.log` at any time.

```
SET user:1 Alice      1714900000000
SET user:2 Bob        1714900001000
DEL user:1            1714900005000
SET session:x Z       1714900010000 EX 30000
```

Why Write WAL Before Memory

The golden rule: If you write to memory first and crash before WAL, the operation is lost permanently — no record exists. If you write WAL first and crash before memory, the WAL record exists and recovery will apply it on next startup. WAL-first guarantees the operation survives any crash at any moment.

Recovery Sequence on Startup

- 1. Read entire WAL file line by line into memory
- 2. Tokenize each line into OPERATION + key + value + timestamp + optional TTL
- 3. Replay each operation against the KVStore in order
- 4. KVStore is now in the exact state it was before the crash
- 5. Truncate or checkpoint the WAL to prevent unbounded growth
- 6. Begin accepting client connections — engine is ready

fsync() — why it is critical: Without `fsync()`, the OS may buffer the write in memory and not flush to physical disk before a crash. `fsync()` forces the OS to flush to storage. This makes individual writes slower but guarantees true durability. The tradeoff between write performance and durability is a real database design decision you can discuss in depth.

9 — Component 6 — Engine Core

What It Is

The Engine class owns all subsystems and provides a single unified interface. The TCP server talks only to the Engine. Nothing else needs to know the internal structure of other components. This is clean dependency management.

Startup Sequence — Order Is Critical

Step	Action	Why Order Matters
1	Replay WAL	Restore KVStore state before anything else runs
2	Start TTLManager	Must come after WAL — janitor cannot delete not-yet-loaded keys
3	ThreadPool ready	Workers started in constructor — already available
4	Wire cross-component events	Subscribe KV expiry events to EventBus
5	Start TCP Server	Only now accept client connections — engine fully ready

Why WAL before TTLManager: If the janitor thread starts before WAL replay completes, it may delete keys that were legitimately set before the crash — because their TTL timestamps have not been loaded from disk yet. Startup ordering is a subtle but critical correctness requirement that demonstrates deep systems thinking.

10 — Build System — CMake

Build Commands

```
# Standard build
mkdir build && cd build
cmake ..
make -j$(nproc)

# Build with ThreadSanitizer -- use for ALL testing
cmake -DTSAN=ON ..
make -j$(nproc)
./tests/stress_test

# Build with AddressSanitizer -- catches memory bugs
cmake -DASAN=ON ..
make -j$(nproc)

# Run all benchmarks
./benchmarks/bench_kvstore
./benchmarks/bench_taskqueue
./benchmarks/bench_pubsub
```

11 — Testing Strategy

Layer 1 — Unit Tests (per component)

- KVStore: set/get/del correctness, TTL before/after expiry, concurrent access
- ThreadPool: all tasks execute, futures resolve, exceptions propagate correctly
- TaskQueue: retry fires after correct backoff, FAILED state after max retries
- EventBus: subscriber receives message, multiple subscribers all notified, unsubscribe works
- WAL: append writes to file, recovery replays all operations in correct order

Layer 2 — Integration Tests

- SET a key, verify WAL entry exists, restart engine, verify key is restored
- Task fails, verify retry fires after correct exponential backoff delay
- KV key expires, verify key.expired event published and received by EventBus
- Full command cycle: TCP client sends SET, engine routes, WAL written, KVStore updated

Layer 3 — Stress Test (Most Important)

- 20 threads running simultaneously for 60 full seconds
- Threads 0-5: random KV SET operations across 10,000 keys
- Threads 6-10: random KV GET operations
- Threads 11-14: submit tasks to thread pool continuously
- Threads 15-17: subscribe and unsubscribe to EventBus topics concurrently
- Threads 18-19: publish messages to EventBus
- Pass condition: zero crashes, zero ThreadSanitizer errors reported

12 — Benchmarking Strategy

Numbers are the language engineers speak. Measured benchmark results in your README prove you actually ran and tested this system. They cannot be faked without running the code. Put them prominently in your README.md and BENCHMARKS.md.

Metric	Expected Target (8-core)	How to Measure
KV read throughput	~2,000,000 ops/sec	16 concurrent reader threads, 5 seconds
KV write throughput	~800,000 ops/sec	16 concurrent writer threads, 5 seconds
KV read latency	~500 nanoseconds avg	chrono::high_resolution_clock per op
TTL precision	±5ms	Measure delay between expiry and actual deletion
Task throughput	~80,000 tasks/sec	8 workers, empty tasks, 5 seconds
Task avg latency	~12 microseconds	Time from submit() call to task start
Pub-Sub throughput	~40,000 msgs/sec	10 topics, 5 subscribers each
Pub-Sub delivery lag	~1 millisecond avg	Timestamp in message vs callback entry time

13 — Phase-by-Phase Build Order

Phase 1 — KV Store	Weeks 1–2
Day 1-2: KVEntry struct + basic KVStore (no threads yet)	
Day 3-4: Add shared_mutex, test concurrent reads and writes	
Day 5-6: Add TTL support — lazy expiry on every GET	
Day 7: Add TTLManager — background janitor thread	
Day 8-9: Write unit tests, run under ThreadSanitizer	
Day 10-11: Write benchmark, measure throughput	
Day 12-14: Demo, documentation, design decisions	
MILESTONE: 1M+ read ops/sec, 30s stress test with zero TSan errors	
Phase 2 — Task Queue	Weeks 3–5
Day 1-2: ThreadPool — basic submit + worker loop	
Day 3-4: Add std::future / packaged_task support	
Day 5-7: Add graceful shutdown sequence	
Day 8-9: Task struct + state machine (PENDING/RUNNING/DONE/FAILED)	
Day 10-11: TaskQueue with retry + exponential backoff	
Day 12-13: Add jitter to backoff	
Day 14-21: Tests, benchmark, demo	
MILESTONE: 10k tasks processed, correct retries, nothing stuck	
Phase 3 — Pub-Sub	Weeks 6–7
Day 1-2: EventBus single-threaded — prove routing logic works	
Day 3-4: Add shared_mutex for concurrent subscribe/publish safety	
Day 5-7: Connect to ThreadPool for fully async dispatch	
Day 8-9: Cross-component: KVStore publishes to EventBus on expiry	
Day 10-14: Tests, benchmark, demo	
MILESTONE: 100 msgs from 5 threads, all 5k subscriber deliveries complete	
Phase 4 — TCP Server	Weeks 8–9
Day 1-2: Basic socket server — accept connections, log them	
Day 3-4: ClientHandler — read command line, send response	
Day 5-7: CommandParser — tokenize and validate all input	

Day 8-10: Connect all commands to Engine.execute()

Day 11-14: Edge cases, test live with telnet and netcat

MILESTONE: Full command cycle from terminal client works end to end

Phase 5 — WAL & Recovery

Weeks 10–11

Day 1-2: WAL append — log SET and DEL operations to file

Day 3-4: Recovery.cpp — replay WAL on startup

Day 5-7: Integration — KVStore calls WAL before every write

Day 8-10: Test crash recovery (kill -9 process, restart, verify data)

Day 11-14: WAL truncation, unit tests, documentation

MILESTONE: Set 1000 keys, SIGKILL process, restart, all 1000 present

Phase 6 — Integration & Polish

Weeks 12–13

Day 1-3: Full stress test with all components running simultaneously

Day 4-5: Final benchmark suite across all five components

Day 6-7: Fix any integration issues found

Day 8-9: Record 5-minute demo video

Day 10-11: Write ARCHITECTURE.md + DESIGN_DECISIONS.md

Day 12-13: Polish README with numbers, build instructions, quick start

MILESTONE: Demo video done, all docs written, git history clean

14 — Key Design Decisions & Why

These are the questions you will be asked in interviews. Every answer demonstrates engineering judgment, not just coding ability. Know all of these cold — they are what separate a good candidate from a great one.

Why `shared_mutex` over `mutex` for `KVStore`?

A regular `mutex` allows only one thread at a time — even for reads. Since reads vastly outnumber writes in a typical cache workload, `shared_mutex` lets all readers proceed simultaneously. Only writes need exclusive access. This can increase read throughput by an order of magnitude.

Tradeoff: `shared_mutex` has higher overhead per operation than `mutex`. For write-heavy workloads a regular `mutex` can actually be faster. Benchmark both — the right answer depends on your access pattern.

Why two-phase deletion in `purgeExpired()`?

Holding a write lock for the full scan blocks all reads for the entire scan duration — potentially 200ms+ on a large store. Scanning under a read lock first lets normal reads continue. The write lock is held only for the brief deletion phase, minimizing latency impact.

Tradeoff: Two-phase means the list of keys-to-delete could be slightly stale. Keys that expire between the two phases are cleaned on the next janitor cycle — this is an acceptable imprecision.

Why exponential backoff with jitter?

Without backoff, a failing task retries immediately in a tight loop, hammering a failing service and making recovery harder. Exponential backoff gives the service time to recover. Without jitter, simultaneous failures all retry at the same moment — thundering herd. Jitter spreads retries in time.

Tradeoff: Tasks take longer to complete when retrying. The delay is intentional — it is protecting downstream services from being overwhelmed.

Why dispatch pub-sub callbacks to a thread pool?

If callbacks run in the publisher's thread, a slow subscriber blocks the publisher and all others. A throwing callback crashes the publisher. Thread pool dispatch isolates each subscriber and makes `publish()` return immediately.

Tradeoff: No guaranteed delivery order between subscribers on the same topic. If order matters, you need a per-subscriber queue or single-threaded dispatch.

Why write WAL before writing to memory?

If you write memory first and crash before WAL, the operation is lost permanently. If you write WAL first and crash before memory, the WAL record exists and recovery applies it on restart. WAL-first guarantees the operation survives any crash at any moment.

Tradeoff: Every write requires an fsync — disk I/O is orders of magnitude slower than memory. This is the fundamental durability vs performance tradeoff every real database must make.

Why one-thread-per-client for TCP?

Simple, correct, and easy to reason about. Each client is completely isolated. No client can block another. Code is straightforward and debuggable — appropriate for a project of this scope.

Tradeoff: Does not scale to thousands of concurrent connections. Production systems use epoll/kqueue event loops. This is an explicit, documented tradeoff — and knowing what you would change is as impressive as the choice itself.

15 — What Every File Does

File	Purpose	Key Concept Demonstrated
KVEntry.h	Data struct for one KV pair	std::optional, std::chrono
KVStore.h/cpp	Thread-safe hash map	shared_mutex, lazy expiry
TTLManager.h/cpp	Background key expiry	atomic, background thread
Task.h	Task struct + state enum	State machine design
RetryPolicy.h	Retry configuration	Backoff parameters, jitter
ThreadPool.h/cpp	Worker thread pool	condition_variable, packaged_task
TaskQueue.h/cpp	Task submission + retry	Exponential backoff, jitter
Message.h	Pub-sub message struct	Data carrier design
EventBus.h/cpp	Topic-based message routing	Observer pattern, async dispatch
TCPServer.h/cpp	Socket accept loop	POSIX sockets, accept()
ClientHandler.h/cpp	Per-client command handling	Read/parse/respond loop
CommandParser.h	Raw string to Command struct	Protocol design
WAL.h/cpp	Disk operation logging	fsync, write-ahead durability
Recovery.cpp	WAL replay on startup	Log-structured recovery
Engine.h/cpp	Wires all components	Dependency injection, startup order
main.cpp	Entry point	Config loading, signal handling
stress_test.cpp	20-thread 60s hammer test	Concurrency correctness proof
bench_kvstore.cpp	Throughput measurement	high_resolution_clock
ARCHITECTURE.md	System diagram + roles	Clear documentation
DESIGN_DECISIONS.md	Why you chose X over Y	Engineering judgment on paper

16 — Interview Preparation

The 90-Second Elevator Pitch

"I built a networked, persistent, multi-model server in C++ from scratch. It combines three subsystems: a thread-safe in-memory key-value store with TTL expiry inspired by Redis, a background task queue with exponential backoff retry inspired by Celery, and a pub-sub messaging system inspired by Kafka. All three are exposed over a TCP server with a text protocol, and all writes are persisted to a write-ahead log for crash recovery. Everything is multithreaded and runs cleanly under ThreadSanitizer."

Questions You Must Be Ready For — Cold, Without Notes

Q: Walk me through a SET command end to end.

Client sends SET user:1 Alice EX 5000 → TCP server reads line → CommandParser tokenizes it → Engine.execute() called → WAL.appendSet() writes to disk with fsync → KVStore.set() acquires unique_lock, stores KVEntry with expiry timestamp computed from now + 5000ms → returns +OK.

Q: How does your TTL work?

Dual strategy. Lazy expiry: every GET checks if the entry's timestamp is before now and returns nullopt if expired. Active expiry: TTLManager runs a background thread waking every 100ms using wait_for on a condition variable, scans all keys, deletes expired ones in two phases. Lazy alone leaks memory. Active alone misses never-read keys. Together they cover both.

Q: What's the hardest bug you fixed?

Write this from your real build journal. Example: 'My purgeExpired held the write lock during the full map scan. Under load with 100k keys this caused 200ms read stalls. I refactored to two-phase: collect expired keys under a read lock, then switch to write lock for deletion only. Read latency dropped by 40% under TTL-heavy load.'

Q: How did you test thread safety?

ThreadSanitizer on every test run — it instruments every memory access at runtime and reports races immediately. Plus a 60-second 20-thread stress test hammering all three subsystems simultaneously. Any race condition shows as a TSan report, not an intermittent crash three weeks later.

Q: What would you do differently?

Replace one-thread-per-client with epoll for thousands of concurrent connections. Add WAL log compaction — it currently grows unboundedly. Add a bloom filter to KVStore to skip memory lookups for nonexistent keys. Add per-topic message queues to pub-sub to guarantee delivery ordering.

Q: How does your thread pool avoid busy-waiting?

Workers block on `condition_variable::wait()` with a predicate checking if the task queue is non-empty OR if the stop flag is set. They consume exactly zero CPU when idle. The predicate also guards against spurious wakeups — condition variables can wake randomly without `notify()` being called.

Final Checklist Before Calling It Done

- ■ All unit tests pass across all 5 components
- ■ Stress test runs 60 seconds clean under `-fsanitize=thread` with zero errors
- ■ Benchmark results measured and written to `BENCHMARKS.md` with real numbers
- ■ 5-minute demo video recorded showing all systems live
- ■ `ARCHITECTURE.md` written with system diagram and component roles
- ■ `DESIGN_DECISIONS.md` written with all 6 decisions and their tradeoffs
- ■ `README.md` has benchmark numbers, build instructions, and quick start guide
- ■ Git history shows incremental commits over weeks — not one single dump
- ■ Can answer every question in this section without referring to notes
- ■ Can run the full end-to-end demo live without preparation

Built from scratch · Every design decision made deliberately · Every tradeoff understood and documented